AgOpenGPS-PGN-Gesamtübersicht

Vollständige technische Zusammenfassung der analysierten PGNs, Protokollpfade und Besonderheiten

Zweck dieses Dokuments

Dieses Dokument fasst die Erkenntnisse aus der Analyse des Repos **AgOpenGPS-Official/AgOpenGPS** zusammen. Der Schwerpunkt liegt auf:

- allen im aktuellen WinForms-Repo gefundenen PGNs,
- ihrem konkreten Aufbau auf Byte-Ebene,
- Datentypen, Byte-Längen, Endianness, Skalierungen und Einheiten,
- Sendern, Empfängern und der tatsächlichen Verwendung im Code,
- dem CRC-/Checksum-Verhalten,
- den Unterschieden zwischen Hauptprotokoll und Discovery-/Management-Pfad,
- sowie Inkonsistenzen und historisch oder implementierungsbedingt gewachsenen Abweichungen.

Das Dokument basiert auf den im Gespräch untersuchten Dateien, insbesondere aus:

- `GPS/Forms/UDPComm.Designer.cs`
- `GPS/Forms/PGN.Designer.cs`
- `GPS/Forms/FormGPS.cs`
- `GPS/Classes/CISOBUS.cs`
- `AgDiag/Protocol/Pgns.cs`
- `AgIO/Source/Forms/UDP.designer.cs`
- `AgIO/Source/Forms/FormUDP.cs`
- `AgIO/Source/Forms/FormLoop.cs`
- `ModSim/Source/Forms/UDP.designer.cs`
- `GPS_Out/Source/PGNs/PGN100.cs`
- `GPS_Out/Source/PGNs/PGN54908.cs`
- `GPS_Out/Source/Classes/UDPcomm.cs`
- `GPS_Out/Source/Classes/clsTools.cs`

---

1. Grundbild: Es gibt praktisch zwei Protokollschichten

Im aktuellen WinForms-Code existieren faktisch **zwei unterschiedliche Kommunikationsschichten**, die zwar denselben äußeren Frame verwenden, aber **nicht dieselbe Strenge bei der CRC-Prüfung** haben.

1.1 Core-AOG-PGN-Protokoll

Dieses Protokoll wird verwendet für:

- GPS-/IMU-Datenübertragung,
- Guidance und Lenksollwerte,
- Rückmeldungen vom Lenkmodul,
- Maschinen-, Hydraulik-, Relay- und Section-Control,
- sowie die ISOBUS-Task-Controller-Kommunikation.

Charakteristik:

- gemeinsamer Byte-Frame,
- additive 8-Bit-Checksumme,
- in der Hauptanwendung (`AgOpenGPS`) wird die Checksumme vor dem Parsen geprüft.

1.2 Discovery-/Management-Protokoll

Dieses Protokoll wird verwendet für:

- Modul-Erkennung,
- Hello-/Heartbeat-Pakete,
- Scan Requests und Scan Replies,
- sowie Subnetz- und IP-Zuweisung.

Charakteristik:

- gleicher äußerer Frame,
- nominell gleiche additive Checksumme,
- in den Discovery-Pfaden von AgIO und ModSim wird CRC aber oft **nicht geprüft**,
- mehrere statische Discovery-Pakete enthalten sogar Endbytes, die **nicht** zur Standard-Checksum-Regel passen.

Praktische Folge:

- **Core-AOG-PGNs** bilden den „strengen“ Protokollpfad,
- **Discovery-/Management-PGNs** einen „lockeren“ Management-/Discovery-Dialekt.

---

2. Allgemeiner PGN-Frame

Alle analysierten AOG-, AgIO- und Modul-PGNs verwenden denselben Grundaufbau:

```
[0x80, 0x81, SRC, PGN, LEN, DATA..., CRC]
```

2.1 Allgemeine Byte-Bedeutung

Byte	Bedeutung
0	Header 1 = `0x80`
1	Header 2 = `0x81`
2	Source Address
3	PGN
4	Payload-Länge
5..N-1	Payload
N	Checksumme

2.2 Endianness

Für mehrbyteige Felder wird im Repo direkt mit `BitConverter` gearbeitet oder per Low-/High-Byte-Zusammensetzung geparst.

Daraus folgt für die On-Wire-Felder im praktischen Sinn:

- `ushort`, `short`, `int32`, `float` und `double` liegen **little-endian** vor,
- bei explizit getrennten `Lo` -/ `Hi` -Bytes gilt ebenfalls little-endian.

2.3 CRC / Checksumme

Die im Repo verwendete „CRC“ ist keine polynomialbasierte CRC, sondern eine **einfache additive 8-Bit-Summe**:

- summiere alle Bytes ab Index `2`,
- bis vor das letzte Byte,
- dann Trunkierung beziehungsweise Modulo 256.

Referenzform:

```
byte crc = 0;
for (int i = 2; i < pgn.Length - 1; i++)
{
    crc += pgn[i];
}
pgn[pgn.Length - 1] = crc;
```

Wichtig:

- im **Core-AOG-Pfad** wird diese Checksumme tatsächlich geprüft,
- im **Discovery-/Management-Pfad** häufig nicht.

---

3. CRC-Verhalten nach Subsystem

3.1 AgOpenGPS-Hauptanwendung (`GPS/Forms/UDPComm.Designer.cs`)

Verhalten:

- Die Länge wird geprüft.
- Die Checksumme wird geprüft.
- Bei Fehler wird das Paket verworfen.

Das ist der strikt implementierte Pfad.

3.2 GPS_Out

Verhalten:

- Die additive Checksumme wird ebenfalls verwendet.

- `clsTools.GoodCRC()` prüft die Checksumme für strukturierte PGN-Pakete.

3.3 AgIO UDP Discovery/Module Path (`AgIO/Source/Forms/UDP.designer.cs`)

Verhalten:

- Oft erfolgt nur die Headerprüfung `0x80 0x81`.
- Es gibt keine generische CRC-Prüfung für Hello-, Scan- und Reply-Traffic.
- Discovery-Pakete werden daher auch mit unpassenden statischen Endbytes akzeptiert.

3.4 ModSim (`ModSim/Source/Forms/UDP.designer.cs`)

Verhalten:

- Keine CRC-Prüfung auf eingehende PGNs.
- Einige ausgehende Pakete berechnen die Checksumme korrekt, etwa `PGN_253` und `scanReply`.
- Mehrere statische Hello- und Config-Pakete haben feste Endbytes, die nicht zur Additivsumme passen.

---

4. Vollständiges PGN-Verzeichnis

Im Folgenden sind alle im Gespräch identifizierten PGNs mit Aufbau, Verwendung und Besonderheiten aufgeführt.

PGN 208 / 0xD0 — Latitude/Longitude Encoded

Status

- Im Repo als Struktur definiert.
- Aktive Verwendung im Hauptfluss nicht belegt.

Richtung

- Unklar; eher Legacy- oder Hilfsstruktur.

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit / Kodierung	Bemerkung
0	Header1	`uint8`	1	-	-	`0x80`
1	Header2	`uint8`	1	-	-	`0x81`
2	Source	`uint8`	1	-	-	`0x7F`
3	PGN	`uint8`	1	-	-	`0xD0`
4	Length	`uint8`	1	-	Byte	`8`
5..8	Latitude Encoded	`int32`	4	little	`lat * (0x7FFFFFFF / 90.0)`	skaliert
9..12	Longitude Encoded	`int32`	4	little	`lon * (0x7FFFFFFF / 180.0)`	skaliert
13	CRC	`uint8`	1	-	additive Summe	

Besonderheiten

- Kein `double`, sondern encoded angle in 32 Bit.
- In der aktuell analysierten Hauptkommunikation nicht der Standard-GPS-PGN.

PGN 211 / 0xD3 — External IMU Data

Richtung

- AgIO / externes IMU-System → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit	Dekodierung / Hinweis
5..6	IMU Heading	`int16`	2	little	`deg * 10`	`* 0.1`
7..8	IMU Roll	`int16`	2	little	`deg * 10`	`* 0.1`
9..10	Angular Velocity	`int16`	2	little	gerätespezifisch	Code dividiert durch `-2`

Verwendung

- Wird in `AgOpenGPS` empfangen und aktualisiert dort IMU-Heading, Roll und Angular Velocity.

Besonderheiten

- Gesamtlänge im Code: 14 Byte.
- Wird im Core-Pfad verarbeitet.

PGN 212 / 0xD4 — IMU Disconnect

Richtung

- AgIO / IMU → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	Disconnect Flag	`uint8`	1	`1` = IMU getrennt

Verwendung

- Setzt interne IMU-Zustände auf Fallback- beziehungsweise Invalidwerte.

Besonderheiten

- Sehr kleines Kontroll-PGN.

PGN 214 / 0xD6 — GPS Position Data

Richtung

- AgIO → AgOpenGPS

Rolle

- Zentrale GPS-, Positions- und Status-Nachricht im Hauptsystem.

Payload-Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit	Dekodierung / Hinweis
5..12	Longitude	`float64`	8	little	Grad	`double`
13..20	Latitude	`float64`	8	little	Grad	`double`
21..24	Heading Dual	`float32`	4	little	Grad	`float.MaxValue` = nicht verfügbar
25..28	Heading True	`float32`	4	little	Grad	Single-Antenna-Heading
29..32	Speed	`float32`	4	little	im Repo nicht formal spezifiziert	Fahrgeschwindigkeit
33..36	Roll	`float32`	4	little	Grad	
37..40	Altitude	`float32`	4	little	m	
41..42	Satellites	`uint16`	2	little	count	
43	Fix Quality	`uint8`	1	-	-	
44..45	HDOP	`uint16`	2	little	HDOP x 100	`*0.01`
46..47	Age	`uint16`	2	little	sec x 100	`*0.01`
48..49	IMU Heading	`uint16`	2	little	`deg * 10`	`*0.1`

Bytes	Feld	Typ	Länge	Endianness	Einheit	Dekodierung / Hinweis
50..51	IMU Roll	`int16`	2	little	`deg * 10`	`*0.1`
52..53	IMU Pitch	`int16`	2	little	gerätespezifisch	direkt übernommen
54..55	IMU Yaw Rate	`int16`	2	little	gerätespezifisch	direkt übernommen

Besonderheiten

- Die Doku nennt 52 Byte Payload.
- Die Hauptanwendung prüft hier die CRC vor der Verarbeitung.
- Dieses PGN bildet die Grundlage für Positionsaktualisierung und Guidance.

PGN 221 / 0xDD — Display Message

Richtung

- Modul / AgIO → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Einheit	Bemerkung
5	Display Time	`uint8`	1	Sekunden	Anzeigedauer
6	Color	`uint8`	1	-	`0 = Salmon`, sonst `Bisque`
7..	Message	UTF-8 String	variabel	-	Textnachricht

Verwendung

- Hardware- oder Statusmeldung im UI.

PGN 222 / 0xDE — Remote Commands

Richtung

- Remote/Modul → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Bemerkung
5	Mask	`uint8`	1	welche Kommandos gültig sind
6	Command	`uint8`	1	Richtung / Kommando

Im Code belegte Bedeutung

- Mask Bit 0: Linie links/rechts nudgen.
- Mask Bit 1: Linienwechsel vor/zurück.

PGN 228 / 0xE4 — Rate Control

Status

- Struktur vorhanden.
- Feldsemantik im aktuellen Repo nur teilweise belegt.

Aufbau

Bytes	Feld	Typ	Länge	Einheit
5	Rate0	`uint8`	1	gerätespezifisch
6	Rate1	`uint8`	1	gerätespezifisch
7	Rate2	`uint8`	1	gerätespezifisch

Besonderheiten

- Weitere Struktur im Code nur angedeutet.
- Keine ausführliche Semantik im aktuell geöffneten Material.

PGN 229 / 0xE5 — Extended Section Control (1..64)

Richtung

- AgOpenGPS → Maschinenmodul

Rolle

- 64-Section- beziehungsweise Zonen-Bitfelder.

Aufbau

Bytes	Feld	Typ	Länge	Einheit / Bedeutung
5	Sections 1..8	`uint8`	1	bitmask
6	Sections 9..16	`uint8`	1	bitmask
7	Sections 17..24	`uint8`	1	bitmask
8	Sections 25..32	`uint8`	1	bitmask
9	Sections 33..40	`uint8`	1	bitmask
10	Sections 41..48	`uint8`	1	bitmask
11	Sections 49..56	`uint8`	1	bitmask
12	Sections 57..64	`uint8`	1	bitmask
13	Tool Left Speed	`uint8`	1	gerätespezifisch
14	Tool Right Speed	`uint8`	1	gerätespezifisch

Verwendung

- Im Hauptsystem als Extended-Section-Control vorhanden.
- In `ModSim` zur Anzeige der acht Bitblöcke genutzt.

Besonderheiten

- Wichtig für 64-Section-Implementierungen.
- Klar konsistent zwischen Hauptcode und ModSim.

PGN 234 / 0xEA – Remote Switches

Richtung

- Modul → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5..12	Switch Data	`byte[8]`	8	Rohdaten / Bitfelder

Verwendung

- Nach Empfang wird `DoRemoteSwitches()` aufgerufen.

Besonderheiten

- Die genaue Bitsemantik liegt außerhalb der im Gespräch geöffneten Stellen.

PGN 235 / 0xEB — Section Dimensions

Richtung

- AgOpenGPS → Maschinenmodul

Rolle

- Breiten der einzelnen Sections.

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit	Bemerkung
5..36	16x Section Width	`uint16` je Section	32	little	`m * 100`	Lo/Hi pro Section
37	Num Sections	`uint8`	1	-	count	

Verwendung

- Wird in `FormGPS.SendRelaySettingsToMachineModule()` befüllt und gesendet.

Besonderheiten

- Maximal 16 Section-Breiten im Standardpaket.
- Bei 64-Section-Bitfeldern getrennt von PGN 229.

PGN 236 / 0xEC — Relay Config

Richtung

- AgOpenGPS → Maschinenmodul

Rolle

- Konfiguration von 24 Pins beziehungsweise Relaiszuordnungen.

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5..28	Pin Config 0..23	`uint8[24]`	24	Konfigurationsbytes

Verwendung

- Kommt aus `setRelay_pinConfig`.
- Wird byteweise aus CSV- beziehungsweise Stringkonfiguration übernommen.

PGN 238 / 0xEE — Machine Config

Richtung

- AgOpenGPS → Maschinenmodul

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung

Bytes	Feld	Typ	Länge	Bedeutung
5	Raise Time	`uint8`	1	gerätespezifische Zeit
6	Lower Time	`uint8`	1	gerätespezifische Zeit
7	Enable Hyd	`uint8`	1	Hydraulik an/aus
8	Set0	`uint8`	1	Config- oder Bitfeld
9	User1	`uint8`	1	frei
10	User2	`uint8`	1	frei
11	User3	`uint8`	1	frei
12	User4	`uint8`	1	frei

Verwendung

- Vom Hauptprogramm gesendet.
- In ModSim gelesen und angezeigt.
- Auch in AgDiag als Parserstruktur vorhanden.

Besonderheiten

- Konsistent zwischen Hauptcode und ModSim.

PGN 239 / 0xEF — Machine Data

Richtung

- AgOpenGPS → Maschinenmodul

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	UTurn	`uint8`	1	UTurn-Status
6	Speed	`uint8`	1	Geschwindigkeit, gerätespezifisch
7	Hyd Lift	`uint8`	1	Hydraulikstatus
8	Tram	`uint8`	1	Tramline-Info

Bytes	Feld	Typ	Länge	Bedeutung
9	Geo Stop	`uint8`	1	Geofence / Stop
11	Sections 1..8	`uint8`	1	bitmask
12	Sections 9..16	`uint8`	1	bitmask

Verwendung

- In ModSim gelesen.
- In AgDiag vorhanden.

Besonderheiten

- `data[10]` bleibt in den offenen Strukturen ungenutzt oder reserviert.

PGN 240 / 0xF0 — ISOBUS Heartbeat

Richtung

- Task Controller → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	Status	`uint8`	1	Bit 0 = Section Control enabled; Bits 1..3 = Num Clients
6	Num Sections	`uint8`	1	`0 = kein Implement / keine Section-Fähigkeit`
7..	Section States	bit-packed bytes	variabel	1 Bit je Section

Verwendung

- `CISOBUS.DeserializeHeartbeat()`

Besonderheiten

- Bei `Num Sections = 0` wird ein Idle-Zustand dargestellt.

PGN 241 / 0xF1 – Section Control Enable Request

Richtung

- AgOpenGPS → Task Controller

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	Enabled	`uint8`	1	`0x01 = enable`, `0x00 = disable`

Verwendung

- `CISOBUS.RequestSectionControlEnabled()`

PGN 242 / 0xF2 – ISOBUS Process Data

Richtung

- AgOpenGPS → Task Controller

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Bedeutung
5..6	Identifier	`uint16`	2	little	Prozessdaten-ID
7..10	Value	`int32`	4	little	Prozesswert

Bekannte Identifier im Code

- `513` = Guidance Deviation
- `397` = Actual Speed
- `597` = Total Distance

PGN 243 / 0xF3 — Field Name

Richtung

- AgOpenGPS → Task Controller

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
4	Length	`uint8`	1	`0 = field closed`, max 248
5..	Field Name	UTF-8 String	variabel	aktiver Feldordnername

Besonderheiten

- Der String wird auf 248 Bytes begrenzt.

PGN 250 / 0xFA — Sensor Data

Richtung

- Modul → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	Sensor Data	`uint8`	1	Rohwert

Besonderheiten

- Im aktuellen Material ist keine weitergehende Feldsemantik dokumentiert.

PGN 251 / 0xFB — AutoSteer Config

Richtung

- AgOpenGPS → Steer Module

Wichtigster Befund

Dieses PGN ist **nicht vollständig harmonisiert** zwischen den verschiedenen Teilen des Repos.

Interpretation in `GPS/Forms/PGN.Designer.cs`

Bytes	Feld	Typ	Bedeutung
5	set0	`uint8`	Config-Bitfeld
6	maxPulse	`uint8`	Max Pulse
7	minSpeed	`uint8`	Speed x 10
8	set1	`uint8`	zweites Config-Byte
9	angVel	`uint8`	zusätzlicher Config-/Flag-Wert

Interpretation in `ModSim/UDP.designer.cs`

Bytes	Feld	Typ	Bedeutung
5	setting0	`uint8`	Bitfeld: InvertWAS, RelayActiveHigh, MotorDirection, SingleInputWAS, Cytron, SteerSwitch, SteerButton, ShaftEncoder
6	PulseCountMax	`uint8`	Max Pulse
7	reserviert / WAS speed	`uint8`	wird ignoriert
8	setting1	`uint8`	Danfoss, PressureSensor, CurrentSensor, UseY_Axis
9	-	-	in ModSim nicht genutzt

Diskrepanz

- Hauptdoku, Hauptstruktur und ModSim sind bei Byte 8/9 nicht vollständig deckungsgleich.
- Wahrscheinlich handelt es sich um eine historisch gewachsene Firmware- oder Simulationssicht gegenüber einer späteren Dokumentation.

Empfehlung

- Für neue Implementierungen Byte 5 und Byte 8 als klare Bitfelder behandeln.
- Byte 9 nur mit Vorsicht interpretieren.

PGN 252 / 0xFC — AutoSteer Settings

Richtung

- AgOpenGPS → Steer Module

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit	Bedeutung
5	Kp	`uint8`	1	-	Gain	proportionaler Verstärkungswert
6	High PWM	`uint8`	1	-	PWM	oberes PWM-Limit
7	Low PWM	`uint8`	1	-	PWM	unteres Fahr-PWM
8	Min PWM	`uint8`	1	-	PWM	Mindest-PWM
9	Counts Per Degree	`uint8`	1	-	counts/deg	WAS-Kalibrierung
10..11	WAS Offset	`uint16`	2	little	counts	Zero-Offset
12	Ackerman	`uint8`	1	-	Prozent / config	

Verwendung

- Im Hauptprogramm gesendet.
- In ModSim gelesen.
- In AgDiag gespiegelt.

Besonderheiten

- ModSim überschreibt lokal `lowPWM` mit `minPWM * 1.2` ; das ist Simulationslogik und keine Änderung des empfangenen Paketformats.

PGN 253 / 0xFD — Steer Module Response

Richtung

- Steer Module → AgOpenGPS

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit	Besonderheit
5..6	Actual Steer Angle	`int16`	2	little	`deg * 100`	`*0.01`
7..8	Heading	`int16`	2	little	`deg * 10`	`9999 = N/A`
9..10	Roll	`int16`	2	little	`deg * 10`	`8888 = N/A`
11	Switch Status	`uint8`	1	-	bitmask	Bit0 = work switch, Bit1 = steer switch, Bit2 teils remote
12	PWM	`uint8`	1	-	raw PWM	

Verwendung

- Die Hauptanwendung empfängt und nutzt das Paket.
- ModSim erzeugt dieses Paket als Antwort auf `PGN 254` .
- AgDiag liest es ebenfalls.

CRC in ModSim

- `PGN_253` wird vor jedem Senden **korrekt neu berechnet**.
- Das ist kongruent mit der Standard-Checksum-Regel.

PGN 254 / 0xFE — AutoSteer Data

Richtung

- AgOpenGPS → Steer Module

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit	Bedeutung
5..6	Speed	`uint16`	2	little	gerätespezifisch	in ModSim mit `*0.1` interpretiert
7	Status	`uint8`	1	-	bitmask / state	Guidance-Status
8..9	Commanded Steer Angle	`int16`	2	little	`deg * 100` in ModSim-Interpretation	Soll-Lenkwinkel
10	Line Distance / XTE	`uint8`	1	-	gerätespezifisch	
11	Sections 1..8	`uint8`	1	-	bitmask	
12	Sections 9..16	`uint8`	1	-	bitmask	

Verwendung

- Zentrale Sollwert-Nachricht für das Lenkmodul.
- ModSim liest daraus Speed, Status, Sollwinkel, XTE und Bitfelder und baut dann `PGN 253`.

Besonderheiten

- Konsistent zwischen Hauptcode, AgDiag und ModSim.

---

5. Discovery-/Management-PGNs

Diese PGNs bilden den locker behandelten Management- und Discovery-Pfad.

PGN 200 / 0xC8 — Hello from AgIO

Richtung

- AgIO → Module

Zweck

- Periodische Hello- beziehungsweise Probe-Nachricht zur Modul-Erkennung.
- AgIO sendet diese Nachricht zyklisch.

Typischer Paketinhalt im Code

```
[0x80, 0x81, 0x7F, 200, 3, 56, 0, 0, 0x47]
```

Wichtige Besonderheit

- Der statische letzte Bytewert passt **nicht** zur Standard-Checksum-Regel.
- Das funktioniert dennoch, weil ModSim im Discovery-Pfad keine CRC prüft.

Modul-Hellos / Heartbeats mit `data[3] == 126 / 123 / 121`

Diese Pakete werden von AgIO als kurze Hello- beziehungsweise Statusrückmeldungen von Modulen akzeptiert.

`data[3] == 126`

- AutoSteer Hello
- AgIO erwartet Länge 11 Byte

`data[3] == 123`

- Machine Hello
- AgIO erwartet Länge 11 Byte

`data[3] == 121`

- IMU Hello

- AgIO erwartet Länge 11 Byte

Wichtige Besonderheiten

- Keine CRC-Prüfung in AgIO vor der Verarbeitung.
- ModSim verwendet statische Pakete mit Endbytes, die rechnerisch nicht zur Standard-Checksumme passen.

PGN 201 / 0xC9 — Set Subnet / IP Prefix

Richtung

- AgIO → Module

Zweck

- Module auf ein neues Subnetz umkonfigurieren.

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	Marker 1	`uint8`	1	`201`
6	Marker 2	`uint8`	1	`201`
7	Subnet 1	`uint8`	1	erstes Oktett
8	Subnet 2	`uint8`	1	zweites Oktett
9	Subnet 3	`uint8`	1	drittes Oktett

Verhalten in ModSim

- Nur wenn zusätzlich `LEN == 5`, `data[5] == 201`, `data[6] == 201`.
- Übernimmt die drei Subnetzbytes.
- Startet danach neu.

Wichtige Besonderheit

- Im aktuellen AgIO-Code wird dafür kein CRC neu berechnet.

- ModSim prüft CRC ebenfalls nicht.

PGN 202 / 0xCA — Scan Request

Richtung

- AgIO → Module

Zweck

- Module im Netz suchen beziehungsweise Discovery triggern.

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
5	Marker 1	`uint8`	1	`202`
6	Marker 2	`uint8`	1	`202`
7	Probe Value	`uint8`	1	aktuell `5`

Verhalten in ModSim

- Erwartet `LEN == 3`, `data[5] == 202`, `data[6] == 202`.
- Sendet anschließend mehrere Scan Replies (`PGN 203`).

Wichtige Besonderheit

- Im aktuellen AgIO-Code ebenfalls statisches Endbyte ohne Recompute.

PGN 203 / 0xCB — Scan Reply

Richtung

- Module → AgIO

Zweck

- Rückmeldung auf einen Scan Request.
- Meldet Modulart, Modul-IP und Subnetz zurück.

Aufbau

Bytes	Feld	Typ	Länge	Bedeutung
2	Source	`uint8`	1	Modultyp
3	PGN	`uint8`	1	`203`
4	Length	`uint8`	1	`7`
5	IP1	`uint8`	1	Modul-IP Oktett 1
6	IP2	`uint8`	1	Modul-IP Oktett 2
7	IP3	`uint8`	1	Modul-IP Oktett 3
8	IP4	`uint8`	1	Modul-IP Oktett 4
9	Subnet1	`uint8`	1	Subnetz Oktett 1
10	Subnet2	`uint8`	1	Subnetz Oktett 2
11	Subnet3	`uint8`	1	Subnetz Oktett 3
12	CRC	`uint8`	1	additive Summe

Modultyp über Source-Byte

Source	Bedeutung
`126`	Steer Module
`123`	Machine Module
`121`	IMU Module
`120`	GPS Module

Wichtige Besonderheit

- ModSim berechnet die CRC hier **korrekt neu**.
- AgIO prüft sie trotzdem nicht explizit vor der Auswertung.

---

6. GPS_Out-spezifische PGNs

Diese PGNs gehören nicht zum zentralen Core-Lenk- beziehungsweise Maschinenpfad, sondern zum Hilfswerkzeug `GPS_Out`.

PGN 100 / 0x64 — Corrected Position / optional Fix2FixHeading

Status

- Im aktuellen Repo nur als Parser beziehungsweise Konsument sichtbar.
- Kein Sender im untersuchten WinForms-Repo gefunden.

Richtung

- AOG → GPS_Out

Variante A — Payload-Länge 16

Bytes	Feld	Typ	Länge	Endianness	Einheit
5..12	Longitude	`float64`	8	little	Grad
13..20	Latitude	`float64`	8	little	Grad

Variante B — Payload-Länge 24

Bytes	Feld	Typ	Länge	Endianness	Einheit
5..12	Longitude	`float64`	8	little	Grad
13..20	Latitude	`float64`	8	little	Grad
21..28	Fix2FixHeading	`float64`	8	little	Grad

Besonderheiten

- `GPS_Out` erkennt diese Nachricht im Dispatcher als zusammengesetzten Wert `0x647F`.
- `Fix2FixHeading` wird nur genutzt, wenn die Extended-Variante vorhanden und die Verbindung gültig ist.
- Intern wird bei ungültigem Heading der Sentinelwert `1000` verwendet.

PGN 54908 / 0xD67C — AGIO Translation Packet

Richtung

- AgIO → GPS_Out

Rolle

- Große GPS- beziehungsweise Status-Nachricht für `GPS_Out`.
- Strukturell sehr nah an `PGN 214 / 0xD6`.

Aufbau

Bytes	Feld	Typ	Länge	Endianness	Einheit
5..12	Longitude	`float64`	8	little	Grad
13..20	Latitude	`float64`	8	little	Grad
21..24	HeadingDual	`float32`	4	little	Grad
25..28	TrueHeading	`float32`	4	little	Grad
29..32	Speed	`float32`	4	little	im Repo nicht formal spezifiziert
33..36	Roll	`float32`	4	little	Grad
37..40	Altitude	`float32`	4	little	m
41..42	Satellites	`uint16`	2	little	count
43	FixQuality	`uint8`	1	-	
44..45	HDOPx100	`uint16`	2	little	HDOP x 100
46..47	Agex100	`uint16`	2	little	sec x 100
48..49	IMUHeading	`uint16`	2	little	`deg * 10`
50..51	IMURoll	`int16`	2	little	`deg * 10`
52..53	IMUPitch	`int16`	2	little	`deg * 10`
54..55	IMUYaw	`uint16`	2	little	gerätespezifisch

---

7. Nicht-AOG-PGN-Klassen in GPS_Out

Im Repo gibt es zusätzlich Klassen mit den Namen:

- `PGN_GGA`
- `PGN_GSA`
- `PGN_RMC`
- `PGN_VTG`
- `PGN_ZDA`

Wichtig:

Diese sind funktional **NMEA-Satzgeneratoren** und keine AOG-Steuer-PGNs.

---

8. Kongruenzen und Diskrepanzen

8.1 Kongruent

CRC-Algorithmus dort, wo wirklich gerechnet wird

Wenn CRC im Code tatsächlich berechnet wird, ist die Logik konsistent bei:

- AgOpenGPS Hauptpfad,
- GPS_Out,
- ModSim bei `PGN_253`,
- ModSim bei `scanReply`.

PGN-Strukturen grundsätzlich konsistent bei

- `0xD6`
- `0xD3`
- `0xD4`
- `0xFD`
- `0xFE`
- `0xFC`
- `0xEF`
- `0xEE`
- `0xE5`
- `0xEB`

- `0xEC`
- `0xF0..0xF3`

8.2 Diskrepanzen

A) Discovery-Pakete mit nicht passendem Endbyte

Mehrere statische Discovery- beziehungsweise Hello-Pakete tragen ein Endbyte, das **nicht** zur Standard-Summenregel passt, zum Beispiel:

- `helloFromAgIO`
- `sendIPToModules`
- `scanModules`
- `helloFromAutoSteer`
- `helloFromMachine`
- `helloFromIMU`
- `PGN_237` in ModSim

B) CRC wird im Discovery-Pfad nicht strikt geprüft

- AgIO Discovery-/UDP-Empfang prüft CRC nicht systematisch.
- ModSim prüft CRC auf eingehende Discovery- und Runtime-Pakete ebenfalls nicht systematisch.

C) PGN 251 ist nicht vollständig harmonisiert

- Die Semantik von Byte 8/9 unterscheidet sich zwischen Hauptstruktur und ModSim.

D) Discovery-/Management-PGNs 200–203 waren in der Hauptdoku kaum sichtbar

- Sie sind real im Code verwendet.
- Sie gehören funktional zur Netz- und Modulverwaltung.
- Sie sollten separat vom Core-AOG-Protokoll betrachtet werden.

---

9. Praktische Schlussfolgerungen für Implementierer

9.1 Wenn ein neues Modul implementiert wird

Dann müssen zwei Ebenen unterschieden werden.

Core-Kompatibilität

Pflicht für:

- Lenkung,
- Maschinensteuerung,
- GPS-/IMU-Daten,
- ISOBUS-Daten.

Anforderungen:

- korrekten AOG-Frame verwenden,
- additive CRC korrekt berechnen,
- sinnvollerweise CRC auch prüfen.

Discovery-Kompatibilität

Pflicht für:

- Scan,
- Hello,
- IP-/Subnetz-Setup.

Anforderungen:

- PGNs 200, 201, 202 und 203 verstehen,
- Modul-Hellos `126/123/121` erzeugen können,
- Discovery-Pakete aus dem aktuellen WinForms-Stack auch dann tolerieren, wenn deren statisches Endbyte nicht der korrekten Summenregel entspricht.

9.2 Robuste Neuimplementierung

Eine saubere neue Implementierung sollte idealerweise:

- überall korrekte additive Checksumme erzeugen,
- im Core-Pfad CRC strikt prüfen,

- im Discovery-Pfad tolerant sein, wenn Kompatibilität zu bestehendem AgIO beziehungsweise ModSim gewünscht ist,
- PGN 251 konservativ behandeln und Byte 8/9 nicht blind nach einer einzigen Quelle interpretieren.

---

10. Kurzfazit

Das Repo enthält **kein einziges homogenes PGN-Protokoll**, sondern praktisch:

1. einen **strengen Core-AOG-Pfad** mit echter CRC-Prüfung,
2. einen **lockeren Discovery-/Management-Pfad**, der denselben Rahmen benutzt, CRC aber oft ignoriert.

Die wichtigste technische Besonderheit der gesamten Analyse ist daher:

> Der gleiche äußere PGN-Frame wird im Repo in verschiedenen Teilprogrammen unterschiedlich streng behandelt.

Für Architektur, Firmware, Testtools und Dokumentation sollte dieser Unterschied ausdrücklich benannt werden.